Candidate Skill Assessment Platform: Full Project & Development Plan

A role-based, stack-based technical assessment platform. The candidate chooses a **role** and a **stack**, takes a randomized test built for that combination, and the system scores performance by **sector** (Strong / Medium / Low) and produces a **role-fit report** for the hiring team.

---

1. Project Overview

1.1 Goal

Let companies evaluate candidates fairly and automatically by:

1. Asking only questions relevant to the candidate's chosen role and stack.
2. Scoring each technical sector separately.
3. Classifying each sector as Strong, Medium or Low using role-specific thresholds.
4. Producing an automatic hiring recommendation, including alternative role suggestions.

1.2 User Types

User	What they do
Candidate	Registers, picks role and stack, takes the test, optionally sees a summary
Recruiter / HR	Creates invitations, views candidate reports, compares candidates
Admin / Content Manager	Manages roles, stacks, questions, blueprints, thresholds
Reviewer (SME)	Reviews and approves new questions before they go live

1.3 Scope

MVP (in scope)

- Candidate registration or invite link
- Role then stack selection
- MCQ, output-prediction and bug-fix (code) questions
- Timed, randomized test with auto-submit
- Sector scoring, tiers, role-fit recommendation
- Admin dashboard: candidate list, report page, question management
- 2 stacks at launch (React, Node.js)

Later (out of scope for MVP)

- Adaptive testing (IRT)
- Video proctoring / AI proctoring
- Percentile benchmarking
- Company multi-tenancy and billing

- Integrations (ATS, Slack, email)

---

2. Core Concepts

2.1 Roles and Stacks

Role	Stacks
Frontend Engineer	React, Next.js, Vue, Angular
Backend Engineer	Node.js, Java Spring, Python (Django/FastAPI), .NET
Full Stack Engineer	MERN, Next.js + PostgreSQL, Java + React
QA / Automation Engineer	Selenium, Cypress, Playwright
Junior / Intern	General fundamentals (JS, logic, basics)

2.2 Sectors

Code	Sector	Covers
`frontend`	Frontend & UI State	DOM, rendering, state management, UI performance
`backend`	Backend & Server Logic	APIs, async, auth, business rules
`database`	Database & Data Layer	Schema, indexing, queries, ORM
`debugging`	Debugging & Edge Cases	Bug-fix, boundary values, defensive coding
`architecture`	System Architecture & Scalability	Caching, patterns, concurrency, modularity
`fundamentals`	Language Fundamentals	JS/Java/Python core concepts, logic

Stack-specific questions are tagged with a `stack`. Shared questions (fundamentals, debugging) have `stack = null` or a `language` tag and are reused across stacks.

2.3 Test Blueprint

A **blueprint** defines how many questions come from each sector, at each difficulty, for a role + stack.

Example: **Frontend Engineer, React, 30 questions, 45 minutes**

Sector	Questions	Difficulty mix (E / M / H)
frontend (React-specific)	12	4 / 5 / 3
fundamentals (JavaScript)	6	2 / 3 / 1
debugging	6 (including 2 code bug-fix)	2 / 2 / 2
architecture	3	1 / 1 / 1
web basics (HTML/CSS)	3	1 / 1 / 1

---

3. Scoring and Recommendation Logic

3.1 Question Scoring

Each question has a `difficulty` and `points`:

Difficulty	Points
Easy	1
Medium	2
Hard	3

- **MCQ (single):** full points if correct, else 0.
- **MCQ (multi):** full points if exactly right; optional partial credit = `(correct selected − wrong selected) / total correct`, minimum 0.
- **Output prediction:** exact match against stored answer.
- **Bug-fix (code):** `points × (passed_tests / total_tests)`.

3.2 Sector Score

```
sector_score % = (earned_points_in_sector / max_points_in_sector) × 100
```

Minimum 5 questions per sector are required for a reliable percentage. If fewer, mark the sector as `INSUFFICIENT_DATA`.

3.3 Tier Classification (per role, stored in DB)

Default cutoffs: **Strong ≥ 75, Medium 50–74, Low < 50.** Each role can override these per sector in `role_profiles`.

3.4 Overall Score (weighted)

```
overall % = Σ (sector_score × sector_weight) / Σ sector_weight
```

Weights come from `role_profiles`. Example for Frontend / React:

Sector	Weight	Strong cutoff	Min acceptable
frontend	0.40	75	60
fundamentals	0.20	70	50
debugging	0.20	70	50
architecture	0.10	60	40
web basics	0.10	60	40

3.5 Role-Fit Engine (data-driven, not hardcoded)

For every role in the system, compute a **fit score** from that role's profile:

```
fit(role) = Σ (min(sector_score, 100) × weight) / Σ weight
hard_fail(role) = any sector with is_mandatory = true and score <
min_acceptable
```

Output:

1. **Primary result:** the role the candidate applied for: `Recommended`, `Borderline` or `Not Recommended`.
2. **Alternative roles:** other roles with `fit ≥ 65` and no hard fail, sorted by fit.
3. **Level:** from overall score of the applied role.

Overall % (no hard fail)	Level
⩾ 85	Senior / Independent
70–84	Mid-Level
55–69	Junior (needs mentorship)
< 55	Not recommended

A hard fail downgrades the result to `Borderline` (or `Not Recommended` if two or more hard fails).

3.6 Reference Implementation (Node.js / TypeScript)

ts

```ts
type SectorScore = { sectorId: number; code: string; pct: number };
type ProfileRow = {
  sectorId: number; weight: number;
  strongCutoff: number; minAcceptable: number; isMandatory: boolean;
};

export function tier(pct: number, strong = 75, medium = 50) {
  if (pct >= strong) return "STRONG";
  if (pct >= medium) return "MEDIUM";
  return "LOW";
}

export function fitForRole(scores: SectorScore[], profile: ProfileRow[]) {
  let weighted = 0, totalWeight = 0, hardFails = 0;
  for (const p of profile) {
    const s = scores.find(x => x.sectorId === p.sectorId);
    const pct = s?.pct ?? 0;
    weighted += pct * p.weight;
    totalWeight += p.weight;
    if (p.isMandatory && pct < p.minAcceptable) hardFails++;
  }
  const fit = totalWeight ? weighted / totalWeight : 0;
  return { fit, hardFails };
}

export function levelFor(fit: number) {
  if (fit >= 85) return "Senior / Independent";
  if (fit >= 70) return "Mid-Level";
  if (fit >= 55) return "Junior (needs mentorship)";
  return "Not recommended";
}

export function recommend(appliedRoleId: number, scores: SectorScore[],
                          allProfiles: Map<number, ProfileRow[]>) {
  const results = [...allProfiles.entries()].map(([roleId, profile]) => ({
    roleId, ...fitForRole(scores, profile),
  }));
  const applied = results.find(r => r.roleId === appliedRoleId)!;
  const verdict =
    applied.hardFails >= 2 ? "NOT_RECOMMENDED" :
    applied.hardFails === 1 ? "BORDERLINE" :
    applied.fit >= 55 ? "RECOMMENDED" : "NOT_RECOMMENDED";
  const alternatives = results
    .filter(r => r.roleId !== appliedRoleId && r.hardFails === 0 && r.fit >= 65
```

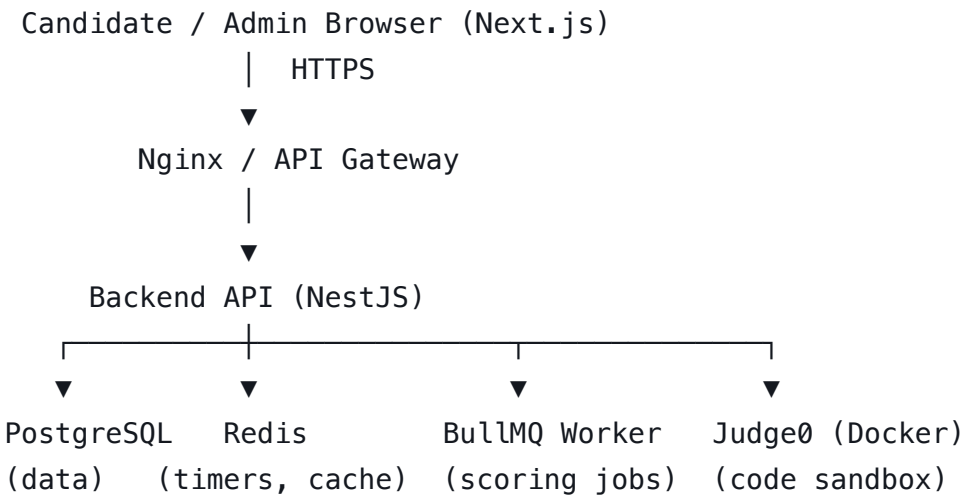
```
    .sort((a, b) => b.fit - a.fit);
  return { verdict, level: levelFor(applied.fit), applied, alternatives };
}
```

4. System Architecture

4.1 Recommended Tech Stack

Layer	Choice	Why
Frontend	Next.js (React) + TypeScript + Tailwind	SSR, routing, fast development
Backend API	Node.js + NestJS (or Express) + TypeScript	Structured, easy to scale
Database	PostgreSQL	Relational data, strong aggregation
ORM	Prisma	Type-safe, migrations
Cache / Sessions	Redis	Test timers, rate limiting, queues
Job Queue	BullMQ (Redis)	Async scoring and code execution
Code execution	Judge0 (self-hosted, Docker)	Secure sandbox for bug-fix questions
Code editor	Monaco Editor	VS Code-like editing in browser
Auth	JWT + refresh tokens (or NextAuth)	Candidate and admin login
File storage	S3-compatible (optional)	Exports, resumes
Email	Resend / SendGrid / SES	Invites and results
Deployment	Docker, Nginx, GitHub Actions CI/CD	Reproducible deploys
Monitoring	Sentry + Prometheus/Grafana (later)	Errors and metrics

4.2 High-Level Architecture



4.3 Test Flow

```
Select role → Select stack → Instructions + consent
      → Start test (server creates session, picks questions via blueprint)
      → Answer questions (autosave every answer)
      → Submit (or timer expires, server auto-submits)
      → Scoring worker: grade → group by sector → tiers → role-fit
      → Save results → Admin sees report
```

Important: the server owns the timer and the answer key. The browser never receives correct answers.

---

5. Database Design (PostgreSQL)

sql

```sql
-- Reference data
CREATE TABLE roles (
  id SERIAL PRIMARY KEY,
  code VARCHAR(50) UNIQUE NOT NULL,        -- 'frontend', 'backend', ...
  name VARCHAR(100) NOT NULL,
  is_active BOOLEAN DEFAULT TRUE
);

CREATE TABLE stacks (
  id SERIAL PRIMARY KEY,
  role_id INT REFERENCES roles(id),
  code VARCHAR(50) NOT NULL,               -- 'react', 'nodejs'
  name VARCHAR(100) NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  UNIQUE (role_id, code)
);

CREATE TABLE sectors (
  id SERIAL PRIMARY KEY,
  code VARCHAR(50) UNIQUE NOT NULL,
  name VARCHAR(100) NOT NULL,
  description TEXT
);

-- Per-role scoring config
CREATE TABLE role_profiles (
  id SERIAL PRIMARY KEY,
  role_id INT REFERENCES roles(id),
  sector_id INT REFERENCES sectors(id),
  weight NUMERIC(4,2) NOT NULL,            -- 0.00 - 1.00
  strong_cutoff INT DEFAULT 75,
  medium_cutoff INT DEFAULT 50,
  min_acceptable INT DEFAULT 50,
  is_mandatory BOOLEAN DEFAULT FALSE,
  UNIQUE (role_id, sector_id)
);

-- Test blueprints
CREATE TABLE test_blueprints (
  id SERIAL PRIMARY KEY,
  role_id INT REFERENCES roles(id),
  stack_id INT REFERENCES stacks(id),
  name VARCHAR(150),
  duration_minutes INT NOT NULL,
```

```
  is_active BOOLEAN DEFAULT TRUE
);

CREATE TABLE blueprint_rules (
  id SERIAL PRIMARY KEY,
  blueprint_id INT REFERENCES test_blueprints(id) ON DELETE CASCADE,
  sector_id INT REFERENCES sectors(id),
  stack_id INT REFERENCES stacks(id),      -- NULL = shared pool
  language VARCHAR(30),                    -- 'javascript' for shared pool
  easy_count INT DEFAULT 0,
  medium_count INT DEFAULT 0,
  hard_count INT DEFAULT 0
);

-- Question bank
CREATE TYPE question_type AS ENUM ('MCQ_SINGLE','MCQ_MULTI','OUTPUT','BUGFIX');
CREATE TYPE difficulty AS ENUM ('EASY','MEDIUM','HARD');
CREATE TYPE review_status AS ENUM ('DRAFT','IN_REVIEW','APPROVED','RETIRED');

CREATE TABLE questions (
  id SERIAL PRIMARY KEY,
  type question_type NOT NULL,
  sector_id INT REFERENCES sectors(id) NOT NULL,
  stack_id INT REFERENCES stacks(id),      -- NULL if shared
  language VARCHAR(30),
  difficulty difficulty NOT NULL,
  points INT NOT NULL,
  title TEXT NOT NULL,
  body TEXT NOT NULL,                      -- markdown, may contain code
  explanation TEXT,
  tags TEXT[],
  status review_status DEFAULT 'DRAFT',
  created_by INT,
  reviewed_by INT,
  times_served INT DEFAULT 0,
  times_correct INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE question_options (
  id SERIAL PRIMARY KEY,
  question_id INT REFERENCES questions(id) ON DELETE CASCADE,
  label CHAR(1),
  text TEXT NOT NULL,
  is_correct BOOLEAN DEFAULT FALSE
```

```
);

CREATE TABLE bugfix_specs (
  question_id INT PRIMARY KEY REFERENCES questions(id) ON DELETE CASCADE,
  language_id INT NOT NULL,                -- Judge0 language id
  starter_code TEXT NOT NULL,
  time_limit_ms INT DEFAULT 2000,
  memory_limit_kb INT DEFAULT 128000
);

CREATE TABLE bugfix_tests (
  id SERIAL PRIMARY KEY,
  question_id INT REFERENCES questions(id) ON DELETE CASCADE,
  input TEXT,
  expected_output TEXT,
  is_hidden BOOLEAN DEFAULT TRUE
);

-- Users
CREATE TABLE users (                       -- admins, recruiters, reviewers
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  role VARCHAR(20) NOT NULL,               -- ADMIN, RECRUITER, REVIEWER
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE candidates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(150) NOT NULL,
  email VARCHAR(255) NOT NULL,
  phone VARCHAR(30),
  experience_years NUMERIC(3,1),
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE invitations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  candidate_email VARCHAR(255) NOT NULL,
  role_id INT REFERENCES roles(id),        -- optional preset
  stack_id INT REFERENCES stacks(id),
  token VARCHAR(100) UNIQUE NOT NULL,
  expires_at TIMESTAMP NOT NULL,
  used_at TIMESTAMP,
  created_by INT REFERENCES users(id)
```

```
);

-- Test sessions
CREATE TYPE session_status AS ENUM ('NOT_STARTED','IN_PROGRESS','SUBMITTED','EX

CREATE TABLE test_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  candidate_id UUID REFERENCES candidates(id),
  blueprint_id INT REFERENCES test_blueprints(id),
  role_id INT REFERENCES roles(id),
  stack_id INT REFERENCES stacks(id),
  status session_status DEFAULT 'NOT_STARTED',
  started_at TIMESTAMP,
  ends_at TIMESTAMP,
  submitted_at TIMESTAMP,
  tab_switch_count INT DEFAULT 0,
  ip_address INET,
  user_agent TEXT
);

CREATE TABLE session_questions (
  id SERIAL PRIMARY KEY,
  session_id UUID REFERENCES test_sessions(id) ON DELETE CASCADE,
  question_id INT REFERENCES questions(id),
  position INT NOT NULL,
  option_order INT[],                    -- shuffled option ids
  UNIQUE (session_id, position)
);

CREATE TABLE responses (
  id SERIAL PRIMARY KEY,
  session_id UUID REFERENCES test_sessions(id) ON DELETE CASCADE,
  question_id INT REFERENCES questions(id),
  selected_option_ids INT[],
  text_answer TEXT,
  code_answer TEXT,
  earned_points NUMERIC(5,2),
  is_correct BOOLEAN,
  time_spent_sec INT,
  answered_at TIMESTAMP DEFAULT NOW(),
  UNIQUE (session_id, question_id)
);

-- Results
CREATE TABLE candidate_sector_analytics (
```

```
  id SERIAL PRIMARY KEY,
  session_id UUID REFERENCES test_sessions(id) ON DELETE CASCADE,
  sector_id INT REFERENCES sectors(id),
  total_questions INT,
  correct_count INT,
  earned_points NUMERIC(6,2),
  max_points NUMERIC(6,2),
  score_percentage NUMERIC(5,2),
  tier VARCHAR(20)                          -- STRONG / MEDIUM / LOW / INSUFFICI
);

CREATE TABLE candidate_recommendations (
  id SERIAL PRIMARY KEY,
  session_id UUID REFERENCES test_sessions(id) ON DELETE CASCADE,
  applied_role_id INT REFERENCES roles(id),
  verdict VARCHAR(30),                      -- RECOMMENDED / BORDERLINE / NOT_RE
  level VARCHAR(50),
  overall_score NUMERIC(5,2),
  alternatives JSONB,                       -- [{roleId, fit}]
  fit_summary TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE audit_events (
  id SERIAL PRIMARY KEY,
  session_id UUID,
  event_type VARCHAR(50),                   -- TAB_BLUR, COPY, PASTE, FULLSCREEN
  meta JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Useful indexes
CREATE INDEX idx_questions_pick ON questions (sector_id, stack_id, difficulty,
CREATE INDEX idx_responses_session ON responses (session_id);
CREATE INDEX idx_sessions_candidate ON test_sessions (candidate_id);
```

---

6. Question Bank Design

6.1 Question Types

Type	Used for	Grading
`MCQ_SINGLE`	Concepts	Option match
`MCQ_MULTI`	"Select all that apply"	Set match / partial
`OUTPUT`	"What does this code print?"	Exact match / normalized
`BUGFIX`	Fix broken code	Hidden unit tests via Judge0

6.2 JSON Import Format

```json
{
  "type": "MCQ_SINGLE",
  "sector": "frontend",
  "stack": "react",
  "language": "javascript",
  "difficulty": "MEDIUM",
  "title": "useEffect dependency array",
  "body": "What happens if `useEffect` has an empty dependency array?",
  "options": [
    { "label": "A", "text": "Runs after every render", "correct": false },
    { "label": "B", "text": "Runs once after the first render", "correct": true
    { "label": "C", "text": "Runs only when props change", "correct": false },
    { "label": "D", "text": "Never runs", "correct": false }
  ],
  "explanation": "An empty array means no dependencies, so the effect runs once
  "tags": ["hooks", "lifecycle"]
}
```

Bug-fix example:

```json
```

```
{
  "type": "BUGFIX",
  "sector": "debugging",
  "language": "javascript",
  "difficulty": "HARD",
  "title": "Fix the off-by-one in pagination",
  "body": "The function returns the wrong items for the last page. Fix it.",
  "starter_code": "function paginate(items, page, size) { ... }",
  "tests": [
    { "input": "[1,2,3,4,5],3,2", "expected": "[5]", "hidden": false },
    { "input": "[1,2,3],1,5", "expected": "[1,2,3]", "hidden": true }
  ]
}
```

6.3 Content Pipeline

1. **Create:** AI-assisted draft, SME-written, or from official docs (never copied from paid or proprietary platforms).
2. **Review:** a developer for that stack verifies the answer key, checks for a single correct option, and checks that wording is unambiguous.
3. **Approve:** only `APPROVED` questions can be served.
4. **Monitor:** track `times_served` and `times_correct`.
    - Correct rate above 95% or below 10% means flag for review or retire.
5. **Rotate:** keep the pool at least 2.5x larger than the number of questions per test.

6.4 Target Volumes

Pool	Target (MVP)
JavaScript fundamentals (shared)	60
Debugging (shared, JS)	40 (including 10 bug-fix)
React-specific	60
Node.js-specific	60
Database (SQL basics + ORM)	40
Architecture (basic)	30
Total MVP	**about 290**

7. Test Generation Algorithm

ts

```ts
async function buildTest(blueprintId: number, candidateId: string) {
  const rules = await db.blueprintRules.findMany({ where: { blueprintId } });
  const picked: number[] = [];

  for (const rule of rules) {
    for (const [level, count] of [
      ["EASY", rule.easyCount], ["MEDIUM", rule.mediumCount], ["HARD", rule.har
    ] as const) {
      if (!count) continue;
      const pool = await db.questions.findMany({
        where: {
          sectorId: rule.sectorId,
          stackId: rule.stackId ?? null,
          language: rule.language ?? undefined,
          difficulty: level,
          status: "APPROVED",
          id: { notIn: picked },
        },
        select: { id: true, timesServed: true },
      });
      // prefer less-served questions, then shuffle for randomness
      const chosen = weightedSample(pool, count, q => 1 / (1 + q.timesServed));
      picked.push(...chosen.map(q => q.id));
    }
  }
  return shuffle(picked);   // final order, grouped or mixed by config
}
```

Rules:

- Throw a clear error if a pool has fewer questions than requested (so admins notice gaps).
- Exclude questions the same candidate saw in a previous attempt.
- Shuffle both question order and option order; store the order in `session_questions` for reproducibility.

---

8. API Design (REST)

Auth

Method	Endpoint	Purpose
POST	`/auth/admin/login`	Admin / recruiter login
POST	`/auth/candidate/start`	Start via invite token or registration
POST	`/auth/refresh`	Refresh JWT

Candidate

Method	Endpoint	Purpose
GET	`/roles`	List roles
GET	`/roles/:id/stacks`	List stacks for a role
POST	`/sessions`	Create session `{roleId, stackId}`
POST	`/sessions/:id/start`	Begin test, sets `ends_at`
GET	`/sessions/:id/questions`	Questions (no answer keys)
PUT	`/sessions/:id/answers/:questionId`	Autosave an answer
POST	`/sessions/:id/run-code`	Run code against visible tests
POST	`/sessions/:id/events`	Log tab switch, blur, paste
POST	`/sessions/:id/submit`	Submit test
GET	`/sessions/:id/status`	Time left, progress

Admin / Recruiter

Method	Endpoint	Purpose
GET	`/admin/candidates`	List with filters (role, stack, score, date)
GET	`/admin/sessions/:id/report`	Full report
GET	`/admin/sessions/:id/report.pdf`	PDF export
POST	`/admin/invitations`	Send invite
CRUD	`/admin/questions`	Manage questions
POST	`/admin/questions/import`	Bulk JSON/CSV import
PATCH	`/admin/questions/:id/status`	Review / approve / retire
CRUD	`/admin/blueprints`	Test blueprints
CRUD	`/admin/role-profiles`	Weights and cutoffs
GET	`/admin/analytics/questions`	Question quality stats

---

9. Frontend Pages

Candidate side

1. **Landing / invite page**: validates token, shows instructions.
2. **Registration**: name, email, phone, experience.
3. **Role selection**: cards for each role.
4. **Stack selection**: options filtered by role.
5. **Instructions + consent**: rules, duration, proctoring notice, system check.
6. **Test screen**: question panel, navigation palette (answered / flagged / unanswered), timer, Monaco editor for code questions, auto-save indicator.
7. **Submission confirmation**: thank-you page (results hidden or summarized, configurable).

Admin side

1. **Login**
2. **Dashboard**: tests today, average score, pass rate by role.
3. **Candidates list**: filter and search, sort by score.
4. **Candidate report**: sector table with tier badges, radar chart, verdict box, alternative roles, answer review, proctoring log.
5. **Compare candidates**: side-by-side sector charts.
6. **Question bank**: list, filters, editor, import, review queue.
7. **Blueprints and role profiles**: edit weights and cutoffs.

8. **Invitations**: create and track.
9. **Analytics**: question difficulty and discrimination stats.

UI details

- Tier badges: green Strong, yellow Medium, red Low.
- Radar chart (Recharts) for sector profile.
- Warn before leaving the test page.
- Fully responsive, but recommend desktop for code questions.

---

10. Security and Anti-Cheating

Platform security

- Never send correct answers to the client; grade only on the server.
- Server-side timer; reject answers after `ends_at`.
- Rate-limit APIs, validate input (Zod / class-validator), parameterized queries via Prisma.
- Run candidate code **only** in Judge0 sandbox (no network, CPU/memory/time limits).
- Hash passwords with bcrypt/argon2; short-lived JWTs; HTTPS only.
- Role-based access control (ADMIN, RECRUITER, REVIEWER).
- Encrypt sensitive candidate data at rest where possible; define a data retention policy (for example, delete after 12 months).

Anti-cheating (practical, privacy-aware)

- Randomized questions and option order from a large pool.
- Per-question time tracking.
- Detect tab switch, window blur, copy/paste, fullscreen exit; log to `audit_events`; show count to recruiters.
- One active session per candidate; one attempt per invite.
- Optional later: webcam snapshots, plagiarism check on code, AI-assisted detection.
- Tell candidates what is monitored (consent screen). This is legally important in many regions.

---

11. Project Structure

```
assessment-platform/
├── apps/
│   ├── web/                    # Next.js frontend
│   │   ├── app/
│   │   │   ├── (candidate)/    # invite, register, role, test
│   │   │   └── (admin)/        # dashboard, candidates, questions
│   │   ├── components/
│   │   └── lib/
│   └── api/                    # NestJS backend
│       └── src/
│           ├── auth/
│           ├── roles/
│           ├── questions/
│           ├── blueprints/
│           ├── sessions/
│           ├── scoring/        # grading, sector analytics, role-fit
│           ├── code-runner/    # Judge0 client
│           ├── reports/
│           ├── invitations/
│           └── common/
├── packages/
│   └── shared/                 # shared types, zod schemas
├── prisma/
│   ├── schema.prisma
│   └── seed.ts
├── content/
│   ├── react.questions.json
│   ├── nodejs.questions.json
│   └── shared-js.questions.json
├── docker-compose.yml          # postgres, redis, judge0, api, web
├── .github/workflows/ci.yml
└── README.md
```

---

12. Development Plan (12 Weeks)

Assumes 1-2 full-stack developers plus part-time content reviewers. Adjust for team size.

Phase 0: Preparation (Week 0 to 1)

- Finalize scope, roles, stacks, sectors, scoring rules.
- Set up repo, monorepo tooling, linting, formatting, Git workflow.
- Docker Compose with PostgreSQL and Redis.
- Design wireframes for the test screen and report page.

- Start content work: assign reviewers; begin drafting the first 100 questions.

Deliverable: repo skeleton, running local environment, wireframes, first question drafts.

Phase 1: Foundation (Week 2 to 3)

- Prisma schema and migrations for all tables in Section 5.
- Seed roles, stacks, sectors, role profiles, one blueprint.
- Auth (admin login, candidate token flow), RBAC.
- Question CRUD and bulk JSON import endpoint.
- Admin UI: login, question list and editor.

Deliverable: admins can log in, import and manage questions.

Phase 2: Test Engine (Week 4 to 5)

- Role and stack selection UI and APIs.
- Test generation algorithm (Section 7).
- Session lifecycle: create, start, autosave, submit, auto-expire (Redis timer + DB check).
- Candidate test UI: question view, palette, timer, flagging.
- Shuffle questions and options; store order.

Deliverable: a candidate can take a full MCQ test end to end.

Phase 3: Scoring and Role-Fit (Week 6 to 7)

- Grading service for MCQ single/multi and output questions.
- Sector aggregation, tier classification, overall score.
- Role-fit engine using `role_profiles` (Section 3.6).
- BullMQ worker for async scoring after submit.
- Unit tests for scoring with fixed fixtures (very important).

Deliverable: results saved to `candidate_sector_analytics` and `candidate_recommendations`.

Phase 4: Code Questions (Week 8)

- Deploy Judge0 via Docker; build the code-runner service.
- Monaco editor integration, "Run" against visible tests, hidden tests on submit.
- Limits: time, memory, no network; queue executions.
- Add bug-fix questions to the pool.

Deliverable: bug-fix questions work and are graded automatically.

Phase 5: Reports and Admin Dashboard (Week 9 to 10)

- Candidate list with filters and sorting.
- Candidate report: sector table, tier badges, radar chart, verdict box, alternative roles, answer review.
- PDF export.
- Invitation creation and email sending.
- Question analytics (correct rate per question).

Deliverable: recruiters can invite, review, compare and export.

Phase 6: Hardening and Anti-Cheating (Week 11)

- Tab-switch/paste/fullscreen event logging and report section.
- Rate limiting, input validation, security review, backups.
- Load test (for example 200 concurrent test sessions with k6).
- Accessibility pass, mobile warnings, browser checks.

Deliverable: production-ready build.

Phase 7: Pilot and Launch (Week 12)

- Deploy to staging, then production (Docker + Nginx + HTTPS).
- Pilot with 20-30 real or internal candidates.
- Review question statistics; retire bad questions; tune thresholds.
- Collect recruiter feedback; fix issues.

Deliverable: MVP launch with React and Node.js stacks.

Post-MVP Roadmap

Priority	Feature
1	Add stacks one at a time (Next.js, Vue, Angular, Java, Python, .NET)
2	Percentile benchmarking ("better than 70% of React candidates")
3	Adaptive difficulty (IRT) to shorten tests
4	Multi-company tenancy, billing, custom blueprints per company
5	ATS and email/Slack integrations
6	Webcam proctoring, plagiarism detection
7	Candidate feedback report (strengths and learning path)

13. Testing Strategy

Level	What	Tools
Unit	Grading, tier, role-fit, test generation	Jest / Vitest
Integration	API + DB flows (create session to submit to score)	Supertest + test DB
E2E	Candidate full journey, admin report	Playwright
Load	Concurrent sessions, code execution queue	k6
Content QA	Answer keys verified by SME	Review checklist
Security	Auth, authorization, injection	OWASP checklist, dependency audit

Key unit-test cases for scoring

- All correct gives 100% and all Strong.
- All wrong gives 0% and all Low.
- Sector with fewer than 5 questions gives `INSUFFICIENT_DATA`.
- Boundary values: exactly 75 is Strong, exactly 50 is Medium, 49.99 is Low.
- A mandatory sector below its minimum gives Borderline.
- Timer expiry auto-submits and scores only saved answers.

---

14. Deployment and DevOps

- **Environments:** local, staging, production.
- **Containers:** web, api, worker, postgres, redis, judge0 (Judge0 needs a Linux host with privileged mode or isolate support; keep it on a separate server in production).
- **CI (GitHub Actions):** lint, type-check, unit tests, build, Docker image push.
- **CD:** deploy to a VPS (DigitalOcean / AWS EC2 / Hetzner) via SSH or a container platform.
- **Backups:** nightly PostgreSQL dumps, tested restores.
- **Secrets:** environment variables via a secrets manager, never committed.
- **Monitoring:** Sentry for errors, uptime checks, log aggregation.

Rough sizing for launch: 2 vCPU / 4 GB for the app and database, plus a separate 2 vCPU / 4 GB for Judge0.

---

15. Risks and Mitigations

Risk	Impact	Mitigation
Poor question quality or wrong keys	Unfair results	SME review, status workflow, analytics to retire bad items
Question leakage	Cheating	Large pool, rotation, randomization, per-candidate exclusion
Code execution abuse	Security	Judge0 sandbox, no network, strict limits, separate host
Unreliable percentages (few questions per sector)	Wrong tiers	Minimum 5 questions, show `INSUFFICIENT_DATA`
Arbitrary cutoffs	Poor hiring decisions	Per-role thresholds in DB, calibrate after the pilot
Content effort underestimated	Delay	Start content in Week 0, launch with 2 stacks only
Privacy / legal (proctoring data)	Compliance	Consent screen, retention policy, minimal data collection
Over-reliance on automation	Bad hires	Position as decision support; humans make the final decision

16. Team and Effort Estimate

Role	Count	Responsibility
Full-stack developer	1-2	API, frontend, scoring, DevOps
UI/UX designer (part-time)	1	Test screen, report, dashboard
Stack SMEs / reviewers (part-time)	2-3	Write and review questions
QA (part-time)	1	Test plans, regression

Estimated effort: about 12 weeks for the MVP with 2 developers, or about 18-20 weeks with 1 developer.

17. Definition of Done for MVP

- A candidate can choose a role and stack and complete a timed, randomized test with MCQ, output and bug-fix questions.
- Scores are computed per sector with tiers and a weighted overall score.
- The report shows the sector breakdown, verdict, level and alternative roles.
- Admins can manage questions, blueprints, role weights and cutoffs without code changes.
- Question statistics are available and bad questions can be retired.
- The system has automated tests for scoring, passes a load test, and is deployed with backups and monitoring.
- At least 2 stacks (React, Node.js) have 100+ reviewed questions each.

---

18. Immediate Next Steps (This Week)

1. Confirm the MVP stacks (suggested: React and Node.js).
2. Initialize the repo and Docker Compose (Postgres, Redis).
3. Write the Prisma schema from Section 5 and seed the reference data.
4. Generate the first 60-100 questions in the JSON format from Section 6.2 and assign a reviewer.
5. Build the question import endpoint and admin question list.

Note: this plan is decision support for hiring. Calibrate thresholds with real candidate data after the pilot, and keep a human in the loop for final decisions.